

Count Binary Strings with Exactly K Adjacent 1's

Problem Statement

Given two integers n and k , count the number of binary strings of length n such that the substring "11" appears exactly k times.

Since the answer can be large, return it modulo:

$10^9 + 7$

Examples

Example 1

Input:
 $n = 3$
 $k = 2$

Output:
1

Explanation:

111

contains two adjacent "11" pairs.

Example 2

Input:
 $n = 5$
 $k = 2$

Output:
6

Valid strings:

```
00111
10111
01110
11100
11101
11011
```

Understanding the Problem

We are counting **adjacent pairs of 1's**, not groups.

Example

```
111
```

Adjacent pairs:

```
1 1 1
^ ^
first pair

  ^ ^
second pair
```

Count = 2

Example

```
1111
```

Adjacent pairs:

```
1 1 1 1
^ ^
  ^ ^
    ^ ^
```

Count = 3

Key Observation

Whenever we place a new bit:

Place 0

```
0 + 0 -> no new pair
1 + 0 -> no new pair
```

Count never changes.

Place 1

```
0 + 1 -> no new pair
1 + 1 -> one new pair formed
```

To know whether placing **1** creates a new pair, we **must know the previous bit**.

This is the entire reason for the **prev** state.

Step 1: Building the Recursion

State Definition

Let

```
dfs(i, cnt, prev)
```

denote:

Number of ways to fill positions **i ... n-1**, if we have already formed **cnt** adjacent **"11"** pairs, and the previous bit is **prev**.

State Variables

i

Current index.

`cnt`

Number of "`11`" pairs already formed.

`prev`

Previous bit.

```
0
or
1
```

What has already been built?

```
0 .... i-1 | i .... n-1
-----|-----
already built | yet to build
```

Base Case

If all positions are filled:

```
if i == n:
```

We have a valid string only if:

```
cnt == k
```

Therefore:

```
if i == n:
    return 1 if cnt == k else 0
```

Pruning

If:

```
cnt > k
```

we can never come back to **k**.

The count only:

```
stays same  
or  
increases
```

Never decreases.

Therefore:

```
if cnt > k:  
    return 0
```

Choices

At every position we have only **2 choices**.

Choice 1: Put 0

```
...0  
...1
```

No new pair is formed.

Transition:

```
dfs(i+1, cnt, 0)
```

Choice 2: Put 1

Two cases.

Case 1

```
prev = 0  
  
0 + 1
```

No new pair.

Transition:

```
dfs(i+1, cnt, 1)
```

Case 2

```
prev = 1  
  
1 + 1
```

One new pair formed.

Transition:

```
dfs(i+1, cnt+1, 1)
```

Recurrence

```
ans = 0  
  
# place 0  
ans += dfs(i+1, cnt, 0)  
  
# place 1  
if prev == 1:  
    ans += dfs(i+1, cnt+1, 1)
```

```
else:  
    ans += dfs(i+1, cnt, 1)
```

Starting State

There is no previous bit initially.

Simplest way:

Choose first bit separately.

```
answer =  
dfs(1,0,0) +  
dfs(1,0,1)
```

Recursive Solution

```
MOD = 10**9 + 7  
  
def solve(n, k):  
  
    def dfs(i, cnt, prev):  
  
        if cnt > k:  
            return 0  
  
        if i == n:  
            return 1 if cnt == k else 0  
  
        ans = 0  
  
        # put 0  
        ans += dfs(i+1, cnt, 0)  
  
        # put 1  
        if prev == 1:  
            ans += dfs(i+1, cnt+1, 1)  
        else:  
            ans += dfs(i+1, cnt, 1)  
  
        return ans % MOD  
  
    return (dfs(1,0,0) + dfs(1,0,1)) % MOD
```

Time Complexity

```
O(2^n)
```

Exponential.

Many overlapping subproblems.

Step 2: Memoization

DP State

```
dp[i][cnt][prev]
```

stores:

```
dfs(i, cnt, prev)
```

Dimensions

i

```
0 → n
```

Size:

```
n+1
```

cnt

$0 \rightarrow k$

Size:

$k+1$

prev

0 or 1

Size:

2

DP Table

```
dp = [[[-1]*2 for _ in range(k+1)]
      for _ in range(n+1)]
```

Memoized Code

```
MOD = 10**9 + 7

def solve(n, k):
    dp = [[[-1]*2 for _ in range(k+1)]
          for _ in range(n+1)]

    def dfs(i, cnt, prev):
        if cnt > k:
            return 0

        if i == n:
            return 1 if cnt == k else 0

    return dfs(0, 0, 0)
```

```
if dp[i][cnt][prev] != -1:
    return dp[i][cnt][prev]

ans = 0

# put 0
ans += dfs(i+1, cnt, 0)

# put 1
if prev == 1:
    ans += dfs(i+1, cnt+1, 1)
else:
    ans += dfs(i+1, cnt, 1)

dp[i][cnt][prev] = ans % MOD
return dp[i][cnt][prev]

return (dfs(1,0,0) + dfs(1,0,1)) % MOD
```

Complexity

States:

$(n+1) * (k+1) * 2$

Time:

$O(n*k)$

Space:

$O(n*k)$

Step 3: Converting Memoization to Tabulation

Golden Rule

Memoization:

```
dp(i, ...)
uses
dp(i+1, ...)
```

Therefore:

```
fill table from bottom to top
```

Table Meaning

```
dp[i][cnt][prev]
```

=

Number of ways to fill positions:

```
i ... n-1
```

given:

- already formed `cnt` pairs
- previous bit is `prev`

Base Row

Recursion:

```
if i == n:
    return 1 if cnt == k else 0
```

Therefore:

```
dp[n][k][0] = 1
dp[n][k][1] = 1
```

All other cells:

```
0
```

Why?

When all positions are filled:

```
cnt == k
```

means one valid string.

Otherwise:

```
0 ways
```

Filling Direction

Recurrence uses:

```
i+1
```

Therefore:

```
for i in range(n-1, -1, -1):
```

Transition for prev = 0

State:

```
dp[i][cnt][0]
```

Choices:

Put 0

```
dp[i+1][cnt][0]
```

Put 1

```
dp[i+1][cnt][1]
```

because:

```
0 + 1
```

does not create a pair.

Therefore:

```
dp[i][cnt][0] =  
dp[i+1][cnt][0] +  
dp[i+1][cnt][1]
```

Transition for prev = 1

State:

```
dp[i][cnt][1]
```

Choices:

Put 0

```
dp[i+1][cnt][0]
```

Put 1

```
dp[i+1][cnt+1][1]
```

because:

```
1 + 1
```

creates one new pair.

Therefore:

```
dp[i][cnt][1] =
dp[i+1][cnt][0]
+
dp[i+1][cnt+1][1]
```

provided:

```
cnt < k
```

Full Tabulation Code

```
MOD = 10**9 + 7

def dp(n,k):
    # states -- i,cnt,prev
    # i--> 0 to n --> size n+1
    # cnt --> we need exactly k pairs, so maximum useful value,0 to k, if it
    # exceeds k,we can stop so size k+1
    # prev can only be 0 or 1, so size 2
    # so the state is dp[i][cnt][prev]
    # how many ways i can fill the remaining positions
    # [0 .... i-1] -> already decided
    # [i .... n-1] -> still need to decide

    # put 0--> dp[i][cnt][prev]+=dp[i+1][cnt][0]
    # if prev==1 --> dp[i][cnt][1]+=dp[i+1][cnt+1][1], else dp[i][cnt][0]+=dp[i+1]
    # [cnt][1]
    dp = [[[0] * 2 for _ in range(k + 1)] for _ in range(n + 1)] # here we fill
    # with 0 because initially 0 ways

    # base row
    dp[n][k][0]=1
    dp[n][k][1]=1

    for i in range(n-1,-1,-1):
        for cnt in range(k,-1,-1):

            # prev=0
```

```

dp[i][cnt][0]=(dp[i+1][cnt][0]+dp[i+1][cnt][1])%MOD

# prev=1
dp[i][cnt][1]=dp[i+1][cnt][0]
if cnt<k:
    dp[i][cnt][1]+=dp[i+1][cnt+1][1]

dp[i][cnt][1]%=MOD

return (dp[1][0][0]+dp[1][0][1])%MOD

```

Why did the if-else disappear?

Memoization:

```

if prev == 1:
    ...
else:
    ...

```

In tabulation we already know the value of `prev`.

We compute separately:

```

dp[i][cnt][0]
dp[i][cnt][1]

```

So the if-else gets unrolled into two formulas.

Pattern Learned Today

This is a classic:

```

Position DP
+
Count DP
+
Previous Character DP

```

Template

```
dp(index, count, prev)
```

Identification Clues

Whenever the problem says:

Exactly K occurrences

Adjacent characters matter

Previous character affects future

Build strings/sequences

Immediately think:

```
dp(index, count, prev)
```

Real World DP Pattern

```
String DP  
+  
Previous State  
+  
Count Constraint
```

General template:

```
dfs(i, count, prev)
```

Questions of this type almost always become:

```
dp[i][count][prev]
```

Similar Problems

Binary Strings with Exactly K Transitions

Count:

```
01  
or  
10
```

pairs.

State:

```
dp(i, transitions, prev)
```

No Consecutive Ones

State:

```
dp(i, prev)
```

Count Strings with Exactly K Occurrences of "AA"

State:

```
dp(i, count, prev)
```

Digit DP Problems

State:

```
dp(pos, count, prev_digit, tight)
```

Count Strings with K Adjacent Equal Characters

State:

```
dp(i, count, prev)
```

Paint Fence Problem

State:

```
dp(i, same_count, prev_color)
```

DP Lessons Learned

Lesson 1

Always build recursion first.

Lesson 2

State must contain only information that affects future.

Lesson 3

Whenever a new choice depends on previous element:

```
include prev in state
```

Lesson 4

Memoization → Tabulation conversion:

1. Write state meaning.
 2. Write recurrence.
 3. Identify dependencies.
 4. Reverse dependency order.
 5. Convert recursive calls into table lookups.
-

Final Template for Similar Problems

```
dfs(index, count, previous_state)
```

If you see:

```
exactly K  
+  
adjacent condition  
+  
strings/sequences
```

this template should immediately come to mind.